# Mixed Precision

## Why Precision Is the Key to Memory Bandwidth, and Why Memory Bandwidth Is the Key to Inference Speed

Mixed precision computing uses lower-precision floating point formats (FP16, BF16) for most operations while strategically keeping certain values in higher precision (FP32). This roughly doubles throughput and halves memory usage on modern GPUs.

This demo covers:

1. Bit layout visualization (FP32, FP16, BF16)
2. Precision loss analysis across value ranges
3. Overflow/underflow boundaries
4. Consecutive value gap (spacing between representable numbers)
5. Dynamic loss scaling dynamics
6. Matrix multiply accumulation error
7. Softmax numerical stability
8. Memory savings at model scale (7B/13B/70B)
9. Training comparison (FP32 vs FP16+scaling vs BF16)

Random seed: 42

*Generated by demo.py*

Number of visualizations: 9

Examples: 9

# Mathematical Foundation: Floating Point Formats

## IEEE 754 Floating Point Representation

Every floating point number is encoded as:

$$\text{value} = (-1)^s \times 2^{e - \text{bias}} \times (1 + m)$$

$s$: sign bit, $e$: exponent field, $m$: mantissa (fraction), bias: exponent offset

## FP32 (Single Precision) -- 32 bits

$[1_{\text{sign}}][8_{\text{exp}}][23_{\text{mant}}] \quad \text{bias} = 127,\ \text{max} \approx 3.4 \times 10^{38},\ \sim 7.2$ decimal digits

## FP16 (Half Precision) -- 16 bits

$[1_{\text{sign}}][5_{\text{exp}}][10_{\text{mant}}] \quad \text{bias} = 15,\ \text{max} = 65504,\ \sim 3.3$ decimal digits

## BF16 (Brain Float) -- 16 bits

$[1_{\text{sign}}][8_{\text{exp}}][7_{\text{mant}}] \quad \text{bias} = 127,\ \text{max} \approx 3.4 \times 10^{38},\ \sim 2.1$ decimal digits

## Consecutive Value Gap

$\text{gap}(v) = 2^{\text{exponent} - m_{\text{bits}}}$ where $m_{\text{bits}}$ is the number of mantissa bits

At $v = 1$: FP32 gap $= 2^{-23} \approx 1.19 \times 10^{-7}$, FP16 gap $= 2^{-10} \approx 9.77 \times 10^{-4}$, BF16 gap $= 2^{-7} \approx 7.81 \times 10^{-3}$

At $v = 1000$: FP16 gap $= 0.5$, BF16 gap $= 4.0 \Rightarrow$ gradient updates $<$ gap are effectively zero

## Precision vs Range Summary

FP16: high precision ($2^{10}$ mantissa values), limited range ($\leq 65504$)

BF16: low precision ($2^7$ mantissa values), full range (same as FP32)

# Mathematical Foundation: Loss Scaling and Numerical Stability

## Loss Scaling for FP16 Training

Small gradients underflow to zero in FP16. Scale the loss before backprop:

$$g_{\text{scaled}} = s \cdot \frac{\partial L}{\partial w} \quad \Rightarrow \quad g = \frac{g_{\text{scaled}}}{s} \quad \text{where } s \text{ is the scale factor}$$

Dynamic scaling algorithm:

1. Start with large $s$ (e.g., $2^{16}$). 2. If overflow (inf/NaN in grads): $s \leftarrow s/2$, skip update.

3. After $N$ clean steps: $s \leftarrow 2s$. Typical $N = 2000$.

## Softmax Numerical Stability

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}} \quad \text{Problem: } e^x \text{ overflows for } x > 11 \text{ in FP16 } (e^{11} \approx 59874 < 65504)$$

$$\text{Solution: softmax}(x)_i = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}} \quad \text{(max exponent is now 0, no overflow)}$$

## Tensor Core Mixed Precision Matmul

$$C = A \cdot B \quad \text{where } A, B \in \text{FP16, accumulator } \in \text{FP32, output } C \in \text{FP16}$$

Pure FP16 accum error $\propto \sqrt{K}$ (inner dim); FP32 accum eliminates this

## Memory Bandwidth Analysis

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes loaded}} \quad \text{GPU is compute-bound when intensity } > \frac{\text{peak FLOPS}}{\text{BW}}$$

$$\text{A100: } \frac{312 \text{ TFLOPS}}{2 \text{ TB/s}} = 156 \text{ FLOP/byte} \quad \text{Decode inference intensity } \approx 1 \text{ FLOP/byte (memory-bound!)}$$

Half precision = half bytes = 2× throughput for memory-bound ops

## Model Memory Formulas

$$\text{Weights: mem} = n_{\text{params}} \times \text{bytes_per_element} \quad \text{7B in FP16: } 7 \times 10^9 \times 2 = 14 \text{ GB}$$

$$\text{KV cache: mem} = 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{seq_len} \times \text{bytes}$$

# Summary of Findings

1. Bit Layout: FP32 uses 8 exponent + 23 mantissa bits. FP16 uses 5+10 (limited range, decent precision). BF16 uses 8+7 (full FP32 range, less precision). BF16 is simply truncated FP32 -- drop the lower 16 bits.

2. Precision Loss: BF16 has ~3x higher mean relative error than FP16, but handles all magnitudes that FP32 can. For neural net weights (clustered near 0), both are adequate.

3. Overflow/Underflow: FP16 overflows at 65504 and underflows below ~5.96e-8. BF16 has the same range as FP32, making overflow/underflow a non-issue. This is why BF16 does not need loss scaling during training.

4. Value Gaps: At value 1000, consecutive FP16 numbers are 0.5 apart and BF16 numbers are 4.0 apart. Gradient updates smaller than this gap vanish -- the staircase effect.

5. Loss Scaling: Dynamic loss scaling grows the scale after N clean steps and halves on overflow. This shifts small gradients into FP16's representable range, preventing the underflow-to-zero problem that stalls training.

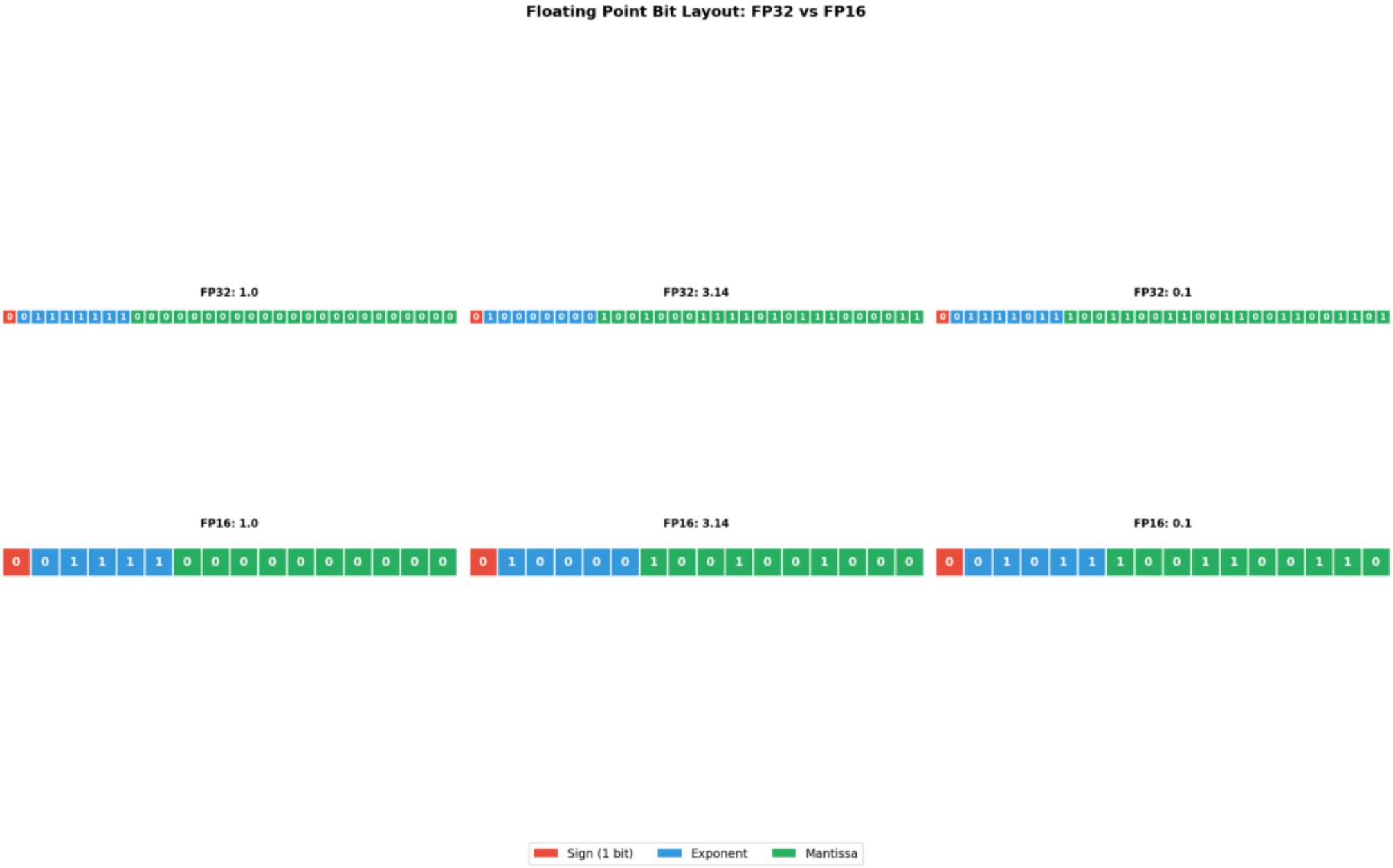
6. Accumulation: Pure FP16 matmul accumulation error grows with inner dimension K. FP32 accumulators (as in tensor cores) eliminate this. Real hardware always uses FP32 accumulation for FP16/BF16 compute.

7. Softmax: exp(x) overflows in FP16 for x > 11. Stable softmax (subtract max) plus FP32 denominator accumulation solves this. All modern frameworks do this by default.

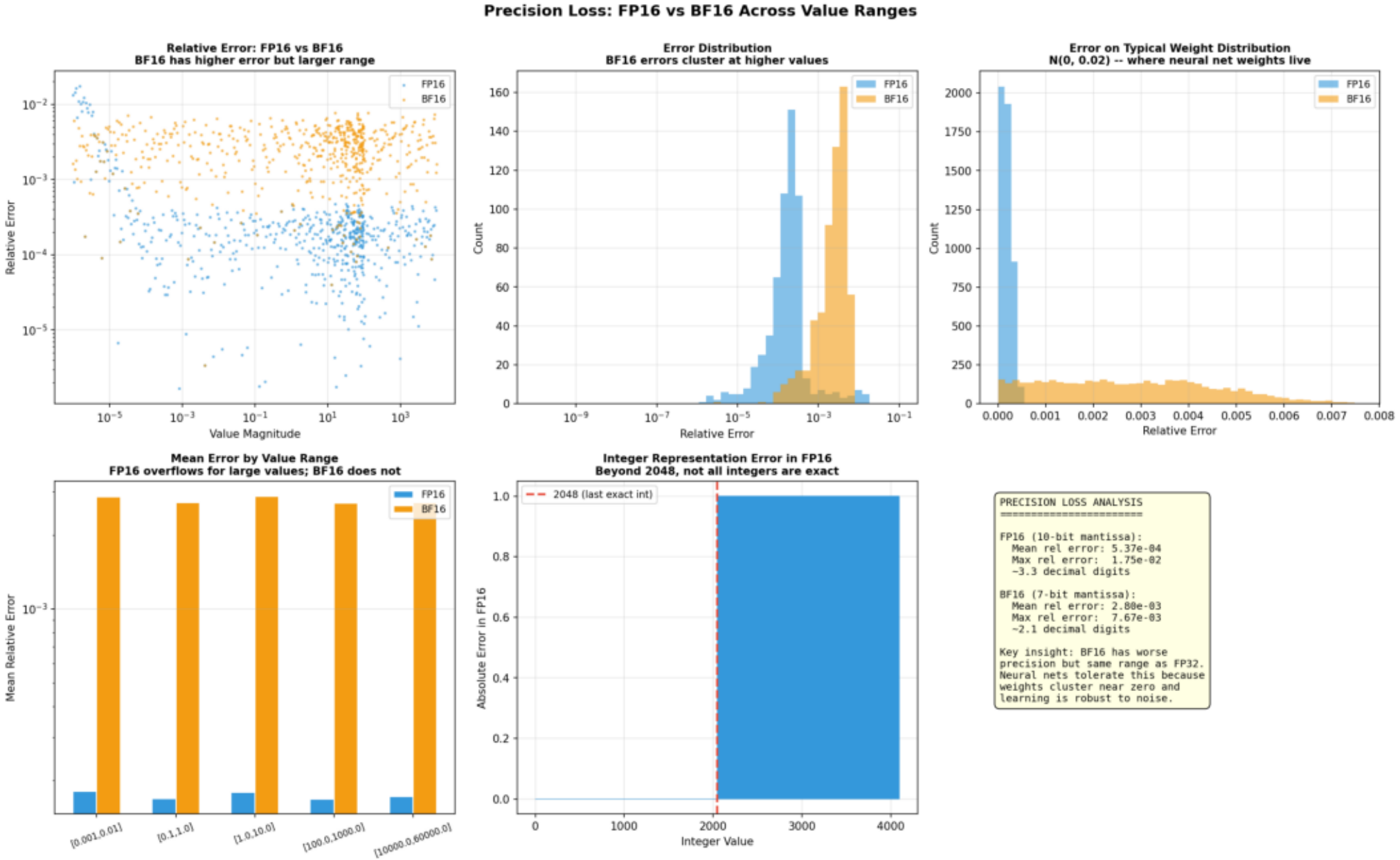
8. Memory: A 70B model in FP32 needs 280 GB (impossible on any single GPU). In FP16/BF16 it's 140 GB (2x H100). In INT4 it's 35 GB (fits on A100 40GB). Precision determines what hardware you need.

9. Training: FP32, FP16+scaling, and BF16 all converge to similar loss on toy problems. BF16 is simplest (no scaling needed). FP16 without scaling works for easy problems but fails when gradients are small.

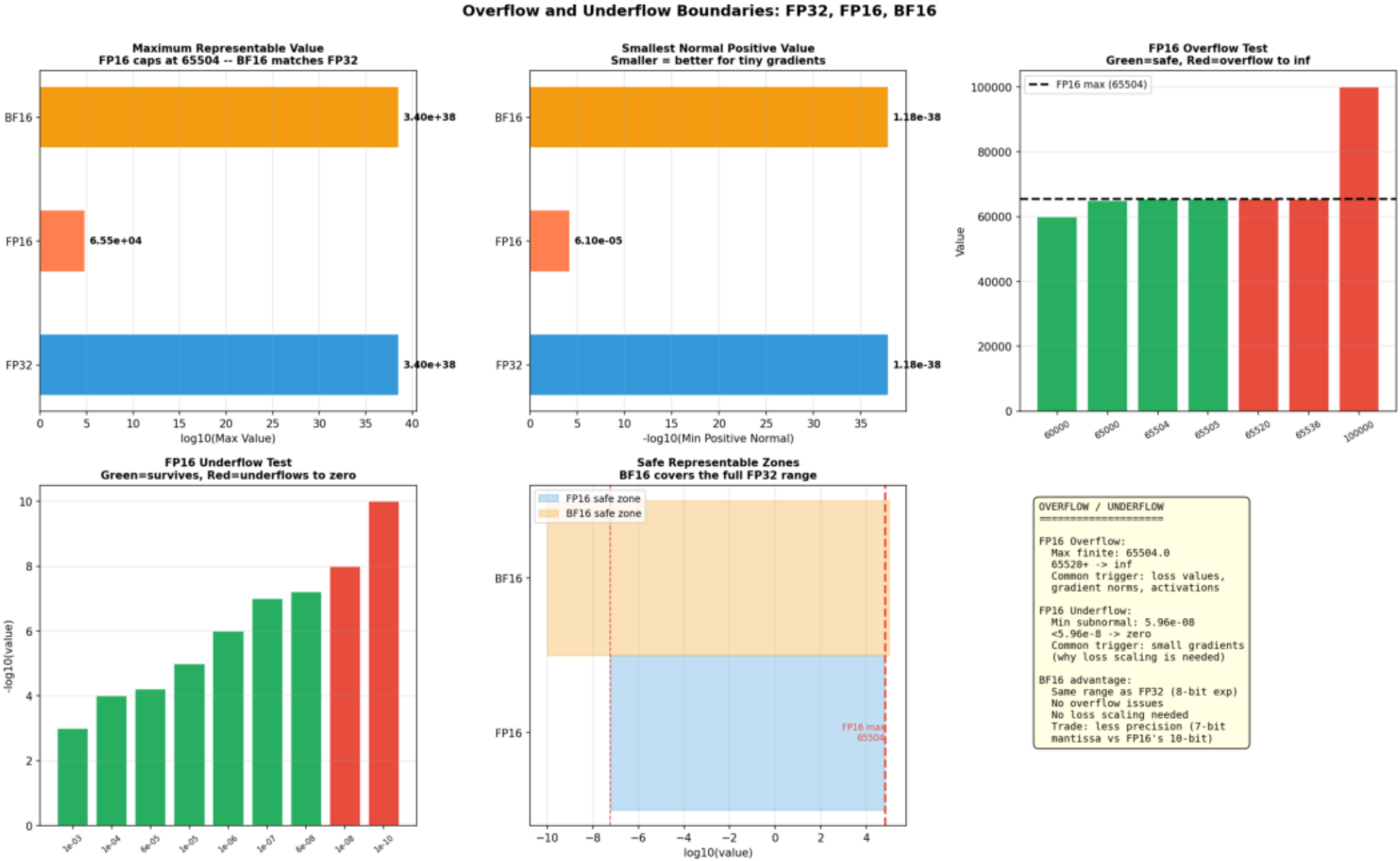
# Example 1: Bit Layout Visualization



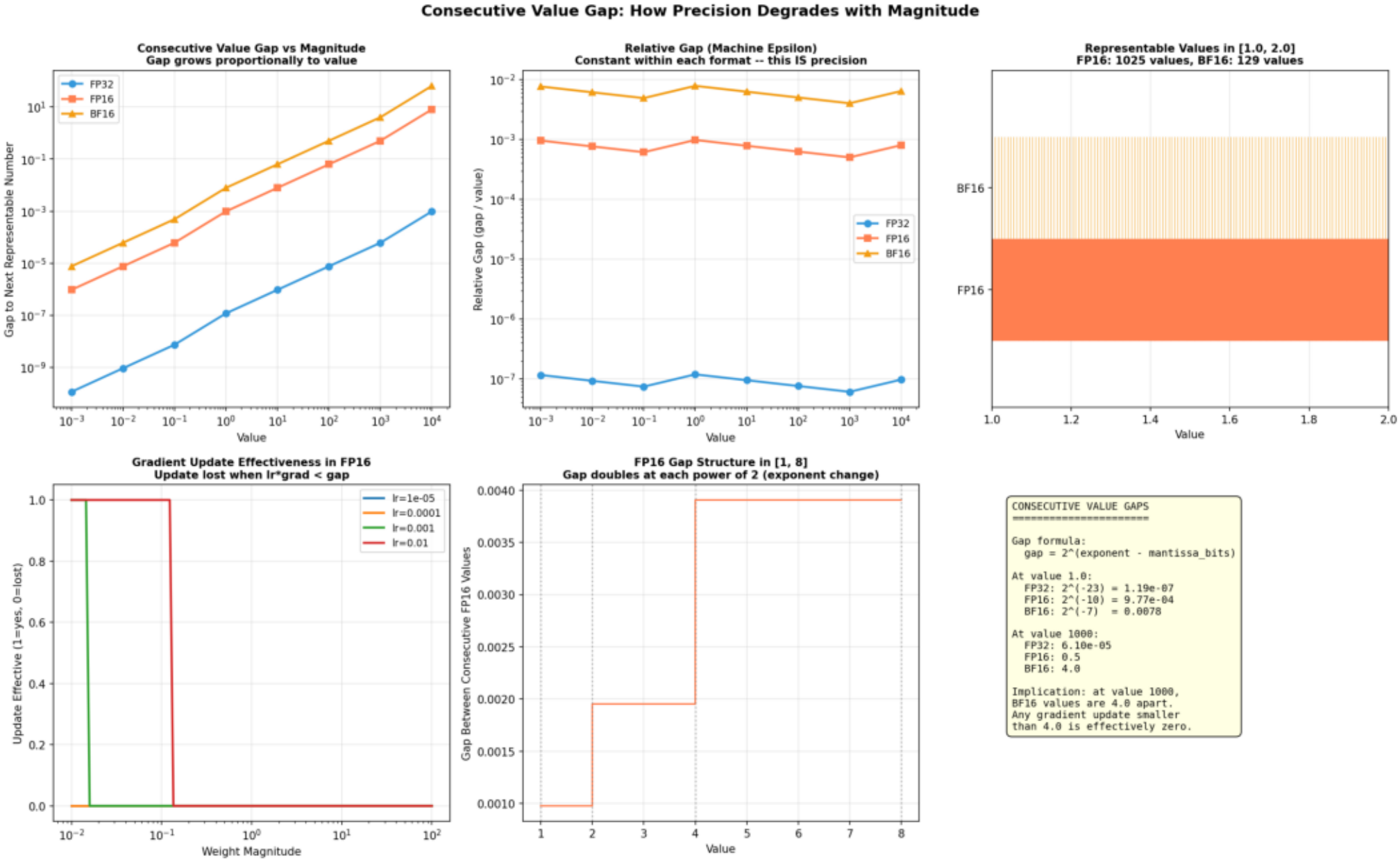
# Example 2: Precision Loss Analysis



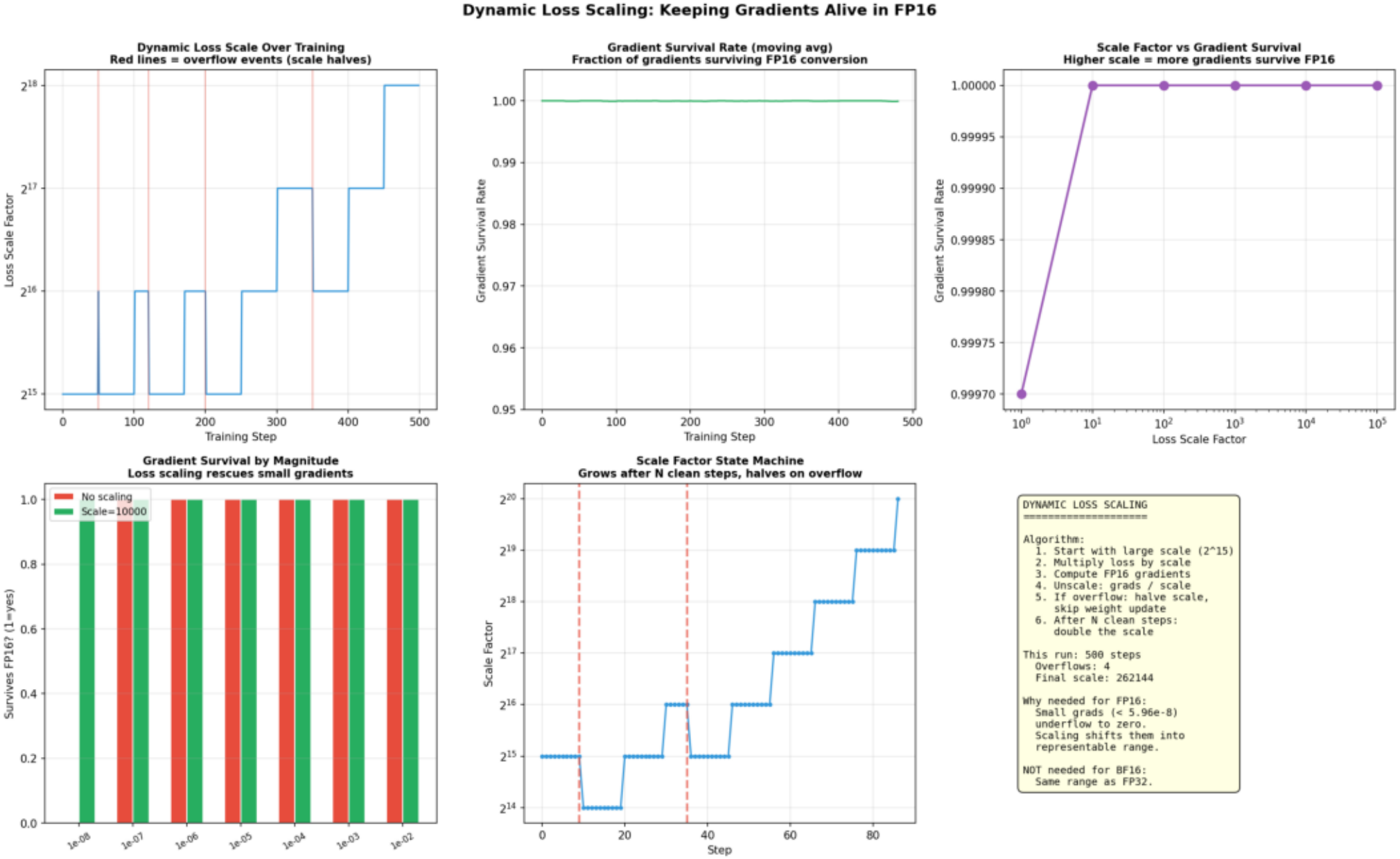
# Example 3: Overflow/Underflow Boundaries



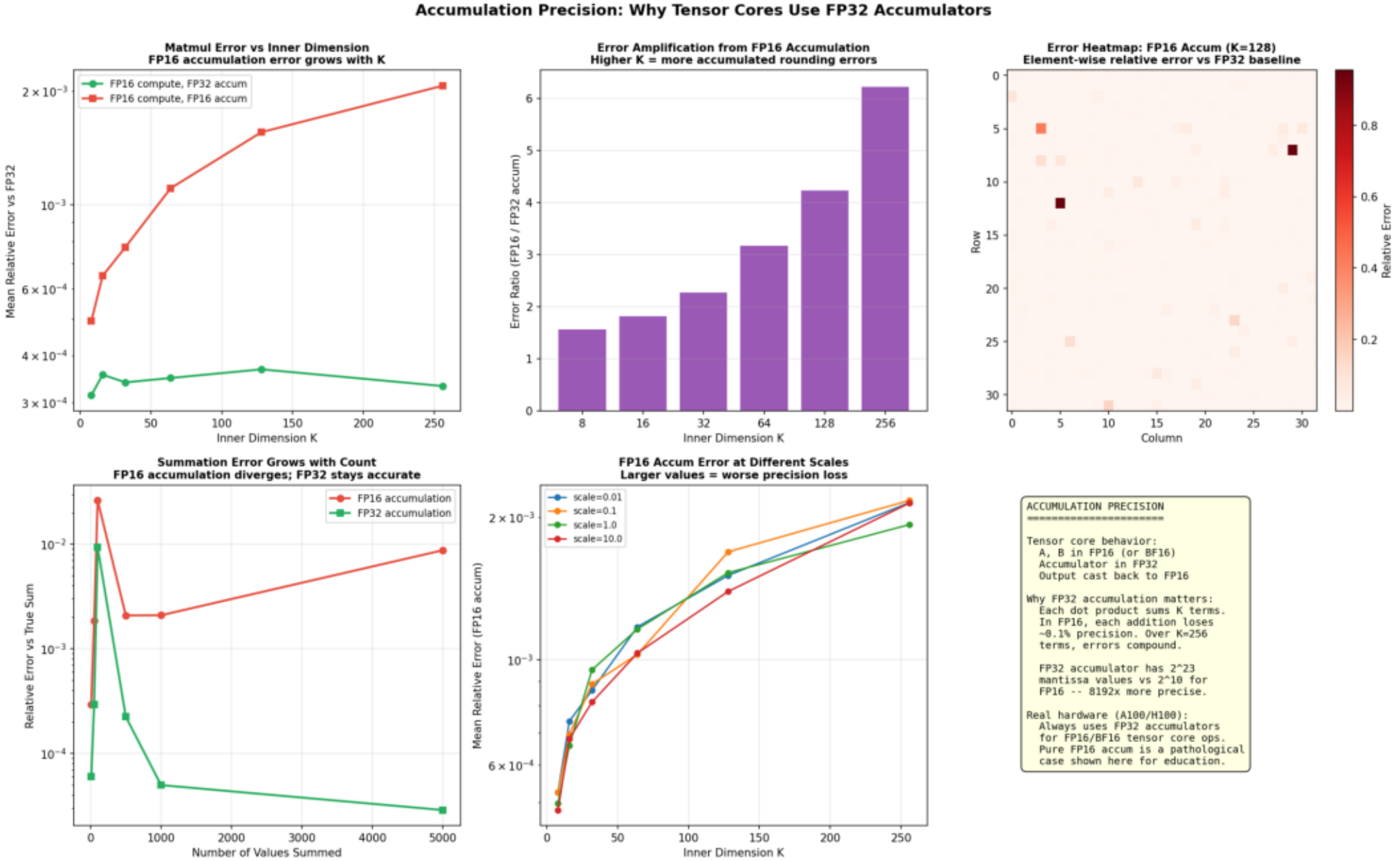
# Example 4: Consecutive Value Gap



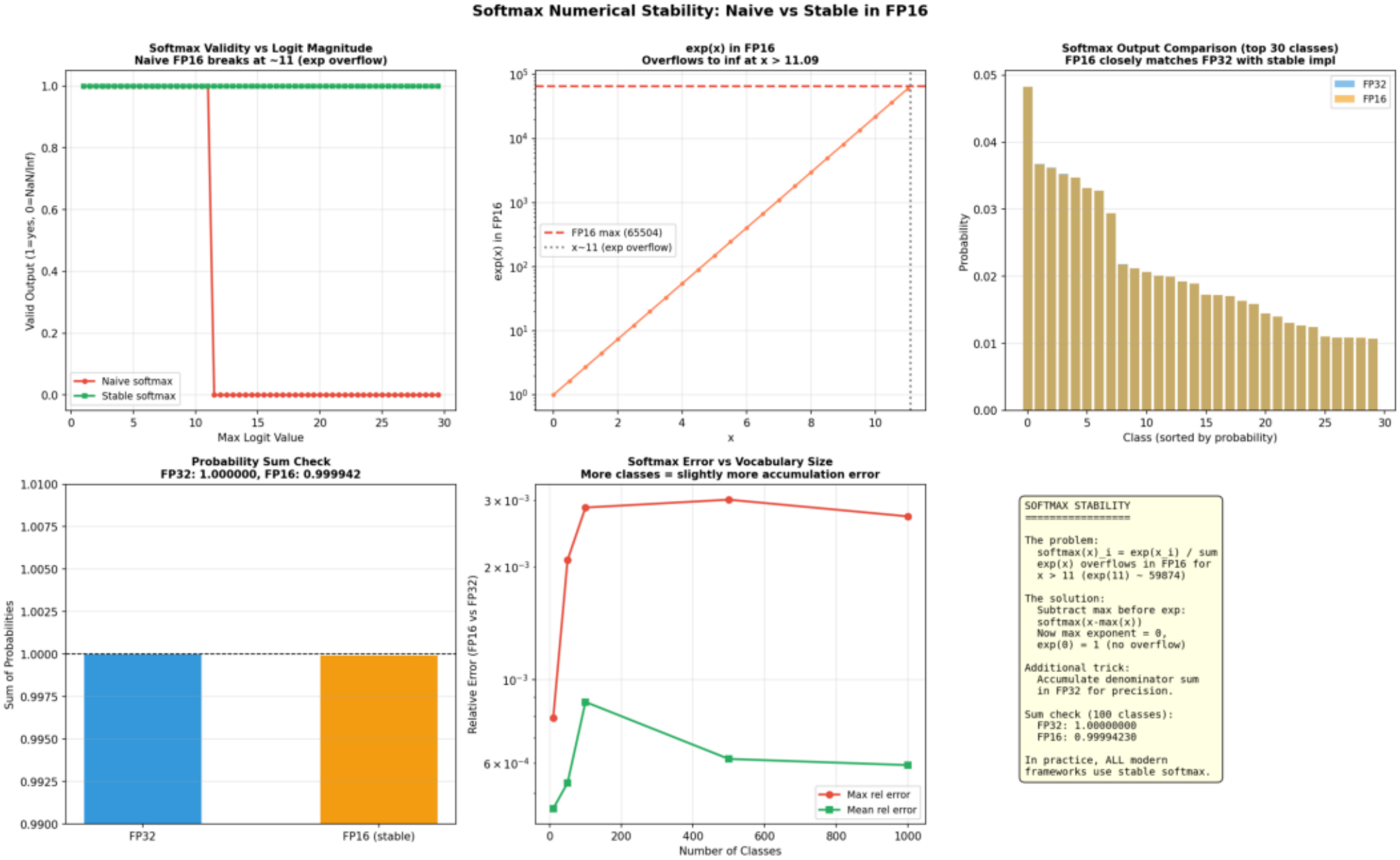
# Example 5: Loss Scaling Dynamics



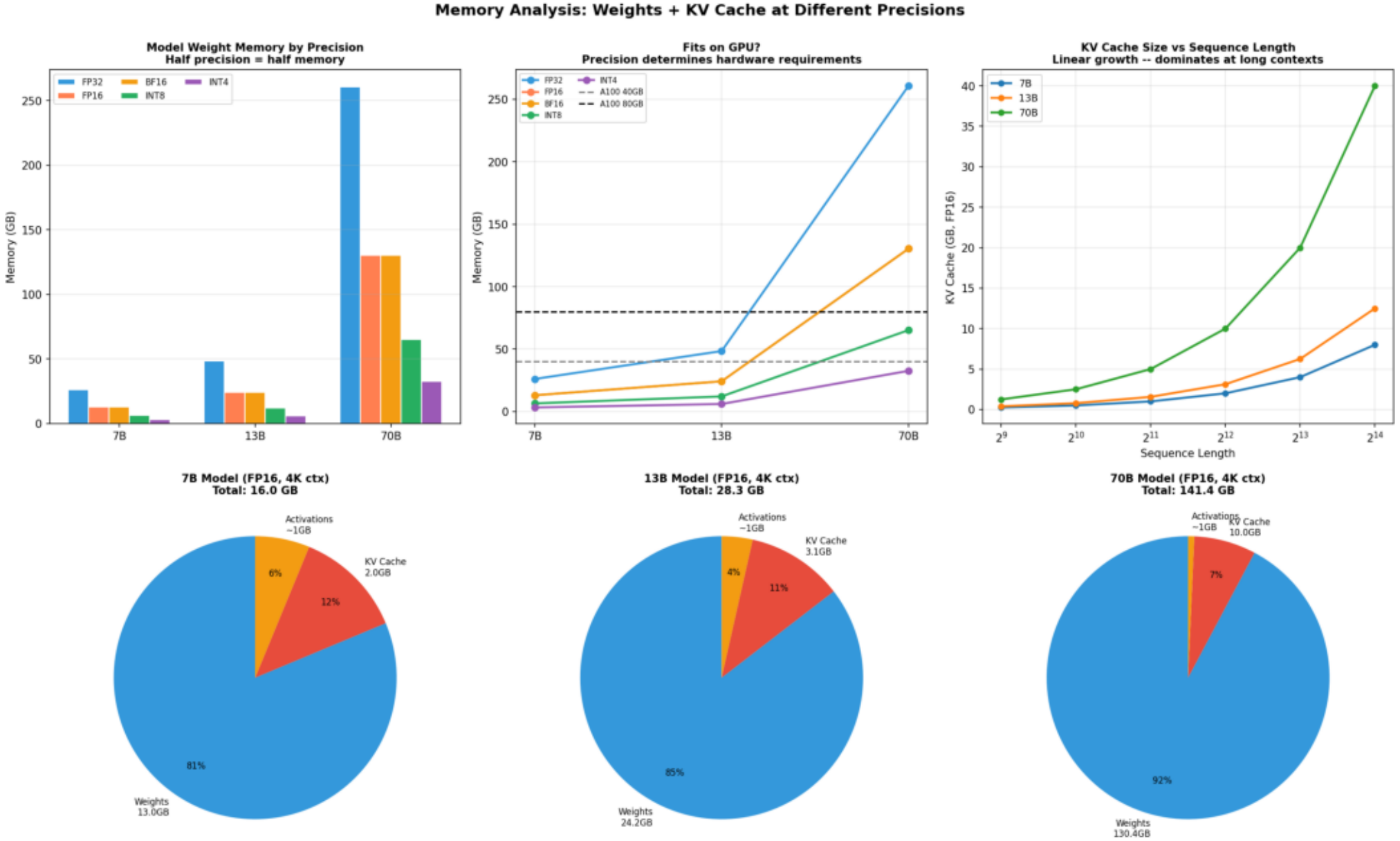
# Example 6: Accumulation Error in Matmul



# Example 7: Softmax Stability in FP16



# Example 8: Memory Savings at Scale



# Example 9: Training Comparison

